

HUBBLE

UZAY TELESKOPU

Data Science Capstone Project Report:

<https://github.com/Selmabanzema/Data-Science-Capston-Project->

[Author: Selmabanzema](https://github.com/Selmabanzema/Data-Science-Capston-Project-)

<https://github.com/Selmabanzema/Data-Science-Capston-Project-/blob/main/jupyter-labs-webscraping.ipynb>

<https://github.com/Selmabanzema/Data-Science-Capston-Project-/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb>

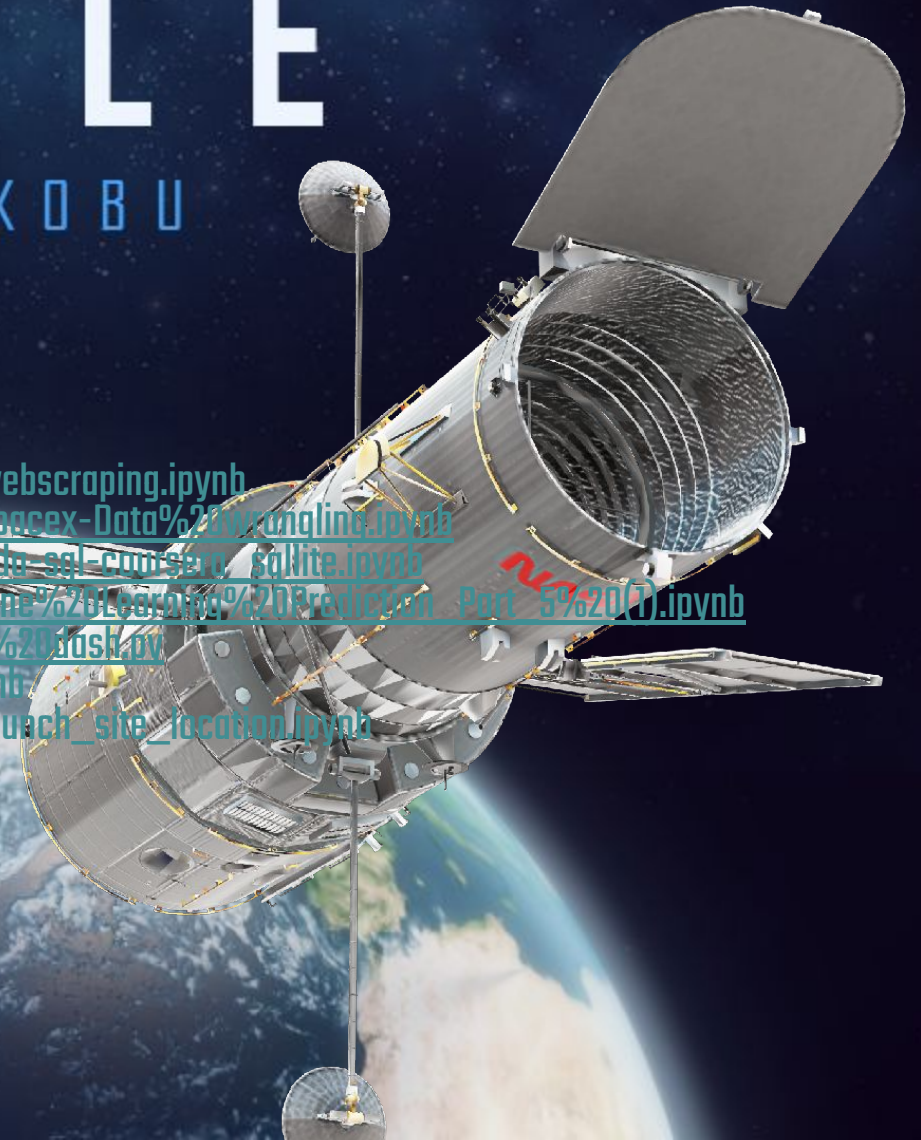
<https://github.com/Selmabanzema/Data-Science-Capston-Project-/blob/main/jupyter-labs-eda-sql-com-sqlite.ipynb>

[https://github.com/Selmabanzema/Data-Science-Capston-Project-/blob/main/Spacex_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/Selmabanzema/Data-Science-Capston-Project-/blob/main/Spacex_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

<https://github.com/Selmabanzema/Data-Science-Capston-Project-/blob/main/creating%20a%20dash.py>

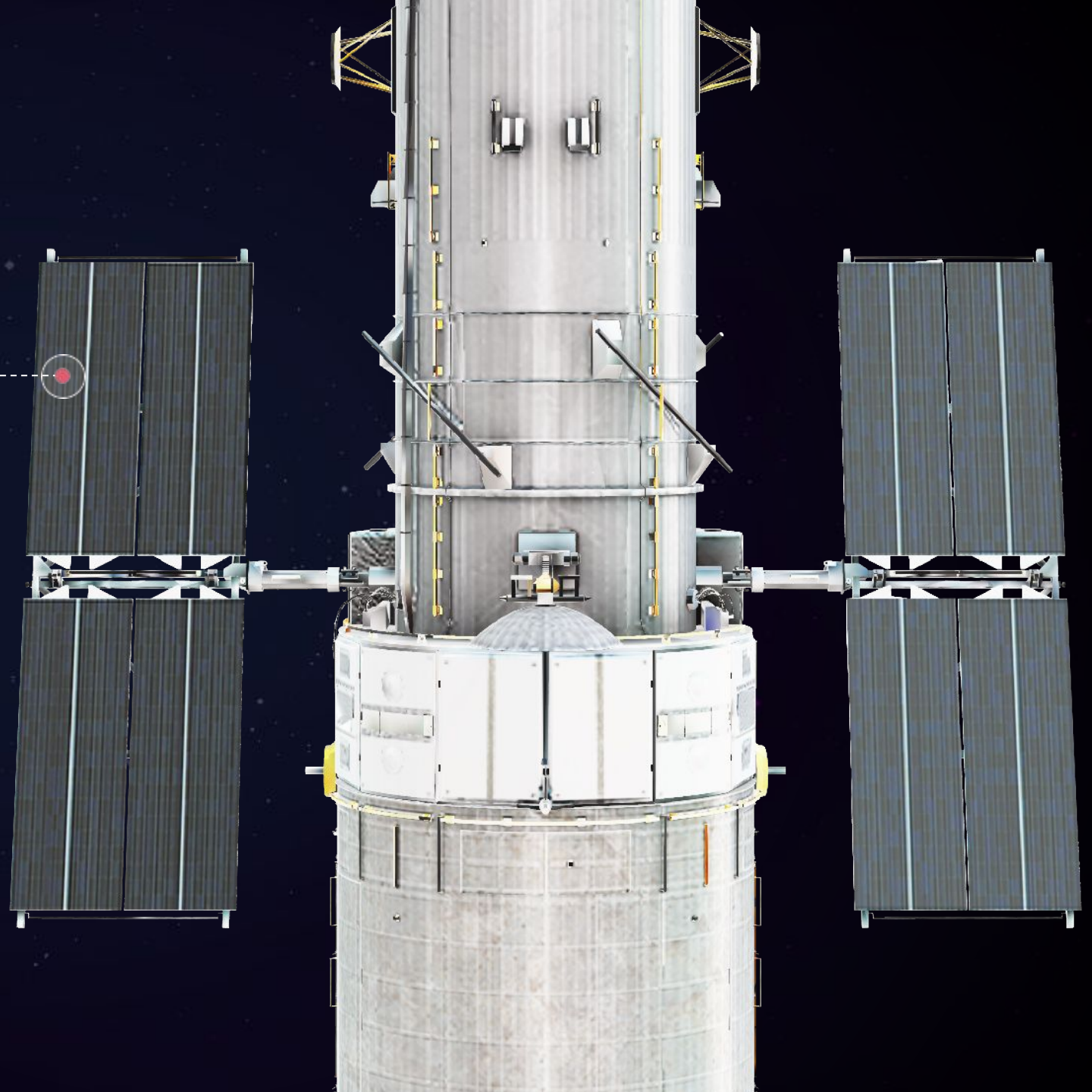
<https://github.com/Selmabanzema/Data-Science-Capston-Project-/blob/main/edataviz.ipynb>

https://github.com/Selmabanzema/Data-Science-Capston-Project-/blob/main/lab_jupyter_launch_site_location.ipynb



AGANDA

- 1- The Main Problem
- 2- TASK1: Data Scrapping And Wraggling
- 3- TASK2: Data Visulization and Dashboard
- 4- TASK3: Data Analysis and EDA
- 5- TASK4: Predective Anaylsis
- 6- Conclusion & Link



THE MAIN PROBLEM : Summary



Falcon 9 is a rocket made by SpaceX that is used to launch things into space, such as satellites and spacecraft.

 Falcon = the name of the rocket family.

Falcon 9 = a specific SpaceX rocket with 9 engines on its first stage.

SpaceX = the company that develops and operates it.

The main problem is will be working on if the station that have been used for falcon will can be reused or note, using pyhon,

Data anyalising, predictive and reporting in this presentation

THE MAIN PROBLEM : Summary



This project investigates SpaceX Falcon 9 launch data with the main objective of understanding and predicting the successful landing of the rocket's first stage. Because first-stage reusability is an important factor in reducing launch costs, predicting landing success can provide valuable information for evaluating launch feasibility and supporting competitive pricing decisions. The project follows a complete data science workflow, beginning with data collection from the SpaceX API and web scraping, followed by data cleaning and transformation. Exploratory analysis was then performed using Python, SQL, statistical analysis, and interactive visualizations to investigate relationships between landing success and factors such as launch site, payload mass, flight number, orbit, and time. Geographical analysis using Folium and an interactive Plotly Dash dashboard provided additional insight into launch-site performance. Finally, several machine-learning classification models were developed and evaluated to predict first-stage landing outcomes. Overall, the project demonstrates how historical launch data can be transformed into actionable insights and predictive information for future Falcon 9 missions.

THE MAIN PROBLEM : Methodology



The project used a structured end-to-end data science methodology. First, **data was collected** using the SpaceX REST API and supplemented with **web scraping** to obtain additional Falcon 9 launch information. The collected data was then **cleaned and wrangled**, including handling missing values, selecting relevant variables, creating the landing-success target variable, and transforming categorical variables into numerical features suitable for analysis and machine learning.

Next, **exploratory data analysis (EDA)** was conducted using Python libraries such as Pandas, Matplotlib, and Seaborn, together with **SQL/SQLite queries** to investigate launch sites, payloads, booster versions, landing outcomes, and historical trends. Statistical and graphical analysis was used to identify relationships between launch characteristics and landing success.

The project then applied **interactive visual analytics**. Folium was used to map SpaceX launch sites and examine their geographical relationship with coastlines, cities, roads, and other features. A **Plotly Dash dashboard** was developed to allow users to interactively filter launch sites and payload ranges and explore landing outcomes.

Finally, the project used **supervised machine learning classification** to predict whether the Falcon 9 first stage would land successfully. Multiple algorithms, including Logistic Regression, Support Vector Machine, Decision Tree, and K-Nearest Neighbors, were trained and evaluated. Hyperparameter tuning was used to improve model performance, and evaluation metrics such as accuracy, F1 score, Jaccard score, and confusion matrices were used to compare predictions. This methodology connects descriptive analysis with predictive modeling and provides a complete framework for understanding Falcon 9 landing performance.

THE MAIN PROBLEM : Key Results



The analysis revealed several important patterns in Falcon 9 landing performance. **Landing success increased over time**, indicating a clear improvement in the historical success rate of first-stage recovery. The exploratory analysis also showed that launch characteristics such as **flight number, launch site, payload mass, and orbit** are associated with differences in landing outcomes. Among the launch sites examined, **KSC LC-39A showed a high level of landing success**, while some orbital categories displayed particularly strong historical success rates. The geographical analysis showed that SpaceX launch sites are positioned close to the **coast**, while also having proximity to transportation infrastructure such as roads and railways, which is relevant to the operational context of launch activities.

The SQL analysis provided another perspective by aggregating the launch records according to payload, booster version, date, and landing outcome. This made it possible to identify historical patterns that are difficult to see from individual launch records alone. The interactive dashboard further transformed these findings into a practical analytical tool, allowing users to investigate how **launch site and payload range relate to successful and unsuccessful outcomes**.

For predictive analytics, multiple classification algorithms were trained and compared. The models produced broadly comparable results on the test data, while the **Decision Tree showed a slight advantage according to the notebook's model-selection criterion**. This indicates that the relationship between launch characteristics and landing outcome can be captured by machine-learning models and used to generate predictions for new launch scenarios. The results should be interpreted as predictions based on historical data rather than guarantees of future landing performance.

Taken together, the project demonstrates that Falcon 9 landing success is not explained by a single variable. Instead, it reflects a combination of **mission characteristics, launch location, payload, booster history, orbit, and technological evolution over time**. An innovative extension of this work would be to turn the predictive model into a scenario-analysis tool: users could change payload, launch site, orbit, and booster characteristics and examine how the predicted landing outcome changes. This would move the project from simply predicting historical outcomes toward supporting **data-driven mission planning and launch-cost analysis**.

TASK1: Data Scrapping



Web Scrapping — Falcon 9 Launch Data

Send an HTTP request

We used `requests.get()` to download the Wikipedia page containing Falcon 9 launch information.

Create a BeautifulSoup object

We used `BeautifulSoup(response.text, "html.parser")` to parse the HTML and make it easier to work with.

Find all tables

We used:

```
html_tables = soup.find_all("table")
```

This finds all `<table>` elements on the webpage.

Find the launch table

We selected the table containing the Falcon 9 launch data and stored it in `first_launch_table`.

Extract column names

We searched for `<th>` elements using `find_all("th")`, then used `extract_column_from_header()` to get each column name.

Store the column names

We added only non-empty column names to the `column_names` list.

In simple terms:

Website → Request HTML → BeautifulSoup → Find tables → Find launch table
→ Extract column names

Link:

<https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/jupyter-labs-webscrapping.ipynb>

TASK1: Data Scrapping: API WORKING FLOW

Task 1: Request and parse the SpaceX launch data using the GET request

To make the requested JSON results more consistent, we will use the following static response object for this project:

```
[9]: static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_
```

We should see that the request was successful with the 200 status response code

```
10]: response=requests.get(static_json_url)
```

```
11]: response.status_code
```

```
11]: 200
```

Now we decode the response content as a JSON using `.json()` and turn it into a Pandas dataframe using `.json_normalize()`

```
12]: # Use json_normalize method to convert the json result into a dataframe
response.json_normalize()
```

Task 2: Filter the dataframe to only include Falcon 9 launches

Finally we will remove the Falcon 1 launches keeping only the Falcon 9 launches. Filter the data to only include Falcon 9 launches. Save the filtered data to a new dataframe called `data_falcon9`.

```
] : # Hint data['BoosterVersion']!='Falcon 1'
```

Now that we have removed some values we should reset the FlightNumber column

```
] : data_falcon9.loc[:, 'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))
data_falcon9
```

Link:

<https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/jupyter-labs-webscraping.ipynb>

Task 3: Dealing with Missing Values

Calculate below the mean for the `PayloadMass` using the `.mean()`. Then use the mean and the `.replace()` function to replace `np.nan` values in the data with the mean you calculated.

```
[ ]: # Calculate the mean value of PayloadMass column
mean=data_falcon9['data_falcon9'].mean()
data_falcon9['data_falcon9'].null.sum()
data_falcon9['data_falcon9'].replace(Nan,mean)
# Replace the np.nan values with its mean value
```

TASK1: Data Scrapping Outputs



```
print(response.content)
```

```
b'<!DOCTYPE html>\n<!--[if lt IE 7]> <html class="no-js ie6 oldie" lang="en-US">\n<![endif]-->\n<!--[if IE 7]> <html class="no-js ie7 oldie" lang="en-US">\n<![endif]-->\n<!--[if IE 8]> <html class="no-js ie8 oldie" lang="en-US">.....
```

```
response.status_code
```

```
20
```

Print the page title to verify if the BeautifulSoup object was created properly

```
<title>List of Falcon 9 and Falcon Heavy launches - Wikipedia</title>
```

```
print(column_names)
```

```
['Flight No.', 'Date and time ( )', 'Launch site', 'Payload', 'Payload mass', 'Orbit', 'Customer',  
'Launch outcome']
```

Link:

<https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/jupyter-labs-webscrapping.ipynb>

TASK1: Data Scrapping



Web Scrapping — Falcon 9 Launch Data

Send an HTTP request

We used `requests.get()` to download the Wikipedia page containing Falcon 9 launch information.

Create a BeautifulSoup object

We used `BeautifulSoup(response.text, "html.parser")` to parse the HTML and make it easier to work with.

Find all tables

We used:

```
html_tables = soup.find_all("table")
```

This finds all `<table>` elements on the webpage.

Find the launch table

We selected the table containing the Falcon 9 launch data and stored it in `first_launch_table`.

Extract column names

We searched for `<th>` elements using `find_all("th")`, then used `extract_column_from_header()` to get each column name.

Store the column names

We added only non-empty column names to the `column_names` list.

In simple terms:

Website → Request HTML → BeautifulSoup → Find tables → Find launch table
→ Extract column names

Link:

<https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/jupyter-labs-webscrapping.ipynb>

TASK1: Data Wrangling



In this exercise, we used data-wrangling techniques to analyze launch data. We used the `.value_counts()` method to determine the number of launches at each launch site and the number of launches for each orbit, while excluding GTO because it is a transfer orbit. We also counted the different mission outcomes and assigned them to the `landing_outcomes` variable. Then, we created the `landing_class` list, assigning 0 to unsuccessful landing outcomes and 1 to successful ones. Finally, we used the `Outcome` and `Orbit` columns to determine how many missions successfully landed on a drone ship and how many launches went to geosynchronous orbit.

Link:

<https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb>

TASK1: Data Wrangling Example Outputs

```
df.isnull().sum()/len(df)*100
```

```
FlightNumber    0.000000
```

```
Date            0.000000
```

```
BoosterVersion  0.000000
```

```
PayloadMass     0.000000
```

```
Orbit            0.000000
```

```
LaunchSite      0.000000
```

```
df['LaunchSite'].value_counts()
```

```
LaunchSite
```

```
CCAFS SLC 40    55
```

```
KSC LC 39A      22
```

```
VAFB SLC 4E     13
```

```
Name: count, dtype: int64
```

```
bad_outcomes=set(landing_outcomes.keys()[[1,3,5,6,7]])bad_outcomes
```

```
{'False ASDS', 'False Ocean', 'False RTLS', 'None ASDS', 'None None'}
```

Link:

<https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb>

TASK2: Complete the EDA with SQL

Our goal is to understand the data and filter and understand it using sqlite codes,

The goal of this analysis was to use **SQL queries and SQLite** to explore the SpaceX launch dataset and extract meaningful information about launch sites, payloads, booster versions, and landing outcomes. The analysis first loaded the SpaceX data into a database and then used SQL queries to investigate important operational questions. It identified the **unique launch sites** and examined sites beginning with "CCA," calculated the **total payload mass carried by NASA (CRS) missions**, and determined the **average payload mass associated with the F9 v1.1 booster version**. The analysis also identified the date of the **first successful ground-pad landing**, examined boosters that successfully landed on drone ships while carrying payloads between **4,000 and 6,000 kg**, and calculated the total number of successful and unsuccessful mission outcomes.

A further highlight was the use of more advanced SQL techniques, including **subqueries, filtering, aggregation, date extraction, and ranking**. The analysis identified booster versions associated with the maximum payload mass and examined **2015 drone-ship landing failures**, including the month, booster version, and launch site. It also ranked landing outcomes between **June 4, 2010 and March 20, 2017**, providing a chronological perspective on the evolution of Falcon 9 landing performance.

Link:

https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/jupyter-labs-eda-sql-coursera_sqlite.ipynb

Link:

TASK2: Complete the EDA with SQL Example Outputs

https://github.com/Selmabanzema/Data-Science-Capston-Project-/blob/main/jupyter-labs-eda-sql-coursera_sqlite.ipynb

Task 1

Display the names of the unique launch sites in the space mission

```
[16]: %sql select * from SPACEXTBL Limit 10
```

```
* sqlite:///my_data1.db  
Done.
```

```
[16]:
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PA
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
%sql select * from SPACEXTBL where Launch_Site like 'CCA%' limit 5;
```

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
[21]: %sql select PAYLOAD_MASS_KG_ from SPACEXTBL where Customer = 'NASA (CRS)'  
* sqlite:///my_data1.db  
Done.
```

```
[21]:
```

PAYLOAD_MASS_KG_
500
677
2296
2216
2395
1808

Task 4

Display average payload mass carried by booster version F9 v1.1

```
[40]: ##sql select avg(PAYLOAD_MASS_KG_) from SPACEXTBL where Customer = 'NASA (CRS)'  
%sql select distinct Launch_Site from SPACEXTBL  
* sqlite:///my_data1.db  
Done.
```

```
[40]:
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

```
[24]: %sql select min(Date) from SPACEXTBL where Mission_Outcome = 'Success'  
* sqlite:///my_data1.db  
Done.
```

```
[24]:
```

min(Date)
2010-06-04

Link:

https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/jupyter-labs-eda-sql-coursera_sqlite.ipynb

TASK2: Complete the EDA with SQL Detailed Outputs

Display 5 records where launch sites begin with the string 'CCA'

```
%sql select * from SPACEXTBL where Launch_Site like 'CCA%' limit 5;
```

Display the total payload mass carried by boosters launched by NASA

```
%sql select PAYLOAD_MASS__KG_ from SPACEXTBL where Customer = 'NASA (CRS)'
```

Display average payload mass carried by booster version F9 v1.1

```
%sql select distinct Launch_Site from SPACEXTBL
```

List the date when the first succesful landing outcome in ground pad was acheived

```
%sql select min(Date) from SPACEXTBL where Mission_Outcome = 'Success'
```

Link:

https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/jupyter-labs-eda-sql-coursera_sqlite.ipynb

TASK2: Complete the EDA with SQL Detailed Outputs

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
[25]: %sql select Booster_Version from SPACEXTBL where Mission_Outcome = 'Success' and PAYLOAD_MASS_KG > 4000 and PAYLOAD_MASS_KG < 6000
* sqlite:///my_data1.db
Done.
[25]: Booster_Version
      F9 v1.1
      F9 v1.1 B1011
      F9 v1.1 B1014
```

Task 7

List the total number of successful and failure mission outcomes

```
[28]: %sql select count(*) from SPACEXTBL where Mission_Outcome = 'Success' union all select count(*) from SPACEXTBL where Mission_Outcome != 'Success'
* sqlite:///my_data1.db
Done.
[28]: count(*)
      98
      3
```

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
[30]: %sql select substr(Date, 6,2),Booster_Version,launch_site from SPACEXTBL where substr(Date,0,5)='2015'
* sqlite:///my_data1.db
Done.
[30]: substr(Date, 6,2)  Booster_Version  Launch_Site
      01             F9 v1.1 B1012    CCAFS LC-40
```

Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
[9]: %sql select Landing_Outcome,count(*) from SPACEXTBL where date between '2010-06-04' and '2017-03-20' group by Landing_Outcome order by count(*) desc
* sqlite:///my_data1.db
Done.
[9]: Landing_Outcome  count(*)
      No attempt    10
      Success (drone ship)  5
      Failure (drone ship)  5
      Success (ground pad)  3
      Controlled (ocean)  3
      Uncontrolled (ocean)  2
      Failure (parachute)  2
      Failed (drone ship)  1
```

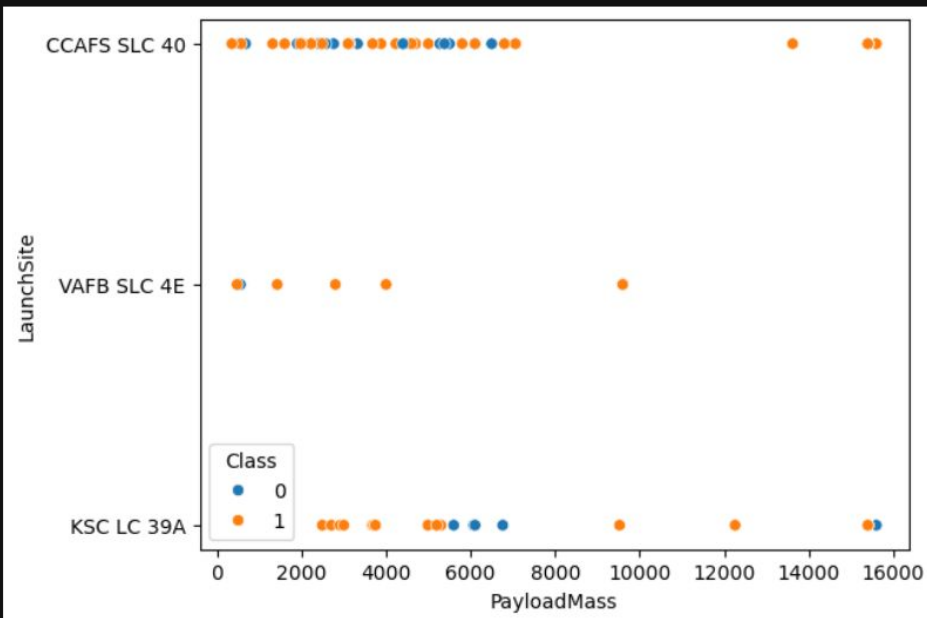
TASK2: EDA with Visualization Lab

The goal of this analysis was to explore and prepare the SpaceX launch dataset for further analysis and machine learning. The dataset was imported and examined using Python, Pandas, Matplotlib, and Seaborn. Visualizations were created to investigate the relationship between **Flight Number**, **Payload Mass**, **Launch Site**, and **Class**, helping to identify patterns between successful and unsuccessful launches. The analysis also included preparing categorical variables by creating **dummy variables** and converting numerical columns to the **float64** data type. These steps provide a clean and structured dataset for subsequent analysis and predictive modeling. Here is example of what we did:

Link:

<https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/edadataviz.ipynb>

```
[7]: <AxesSubplot:xlabel='PayloadMass', ylabel='LaunchSite'>
```



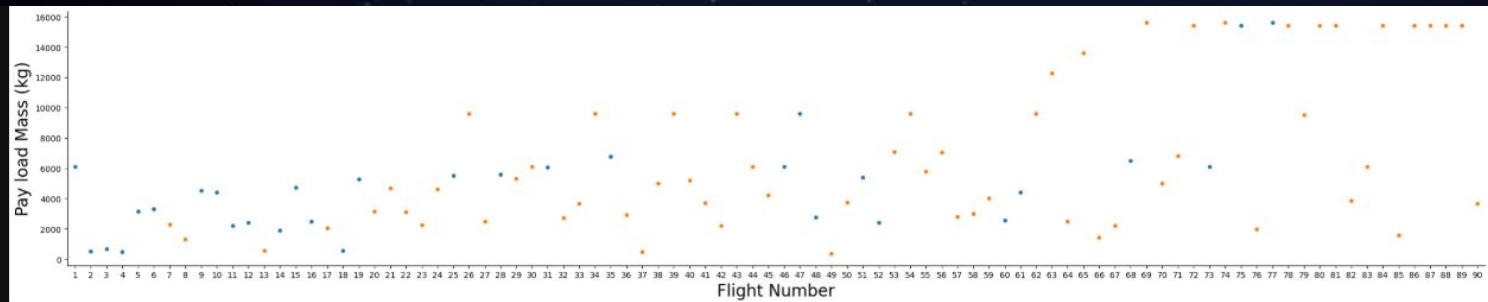
TASK2: EDA with Visualization Lab Example

Outputs

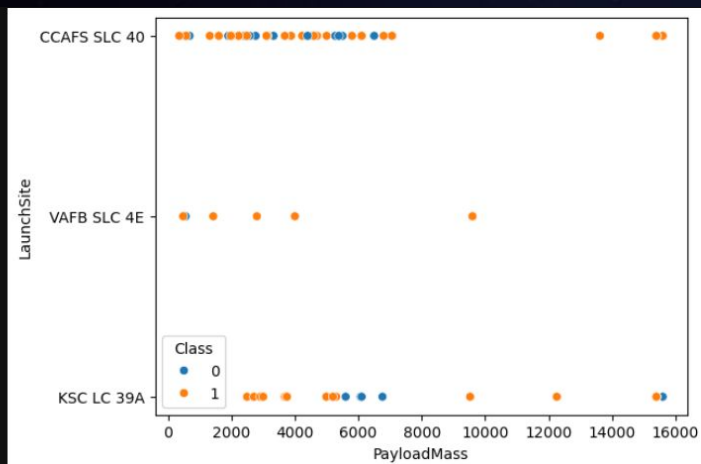
```
sns.catplot(y="PayloadMass", x="FlightNumber", hue="Class", data=df, aspect =  
5)plt.xlabel("Flight Number",fontsize=20)plt.ylabel("Pay load Mass  
(kg)",fontsize=20)plt.show()
```

Link:

<https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/edadataviz.ipynb>



```
sns.scatterplot(x='PayloadMass',y='LaunchSite',data=df,hue='Class')
```

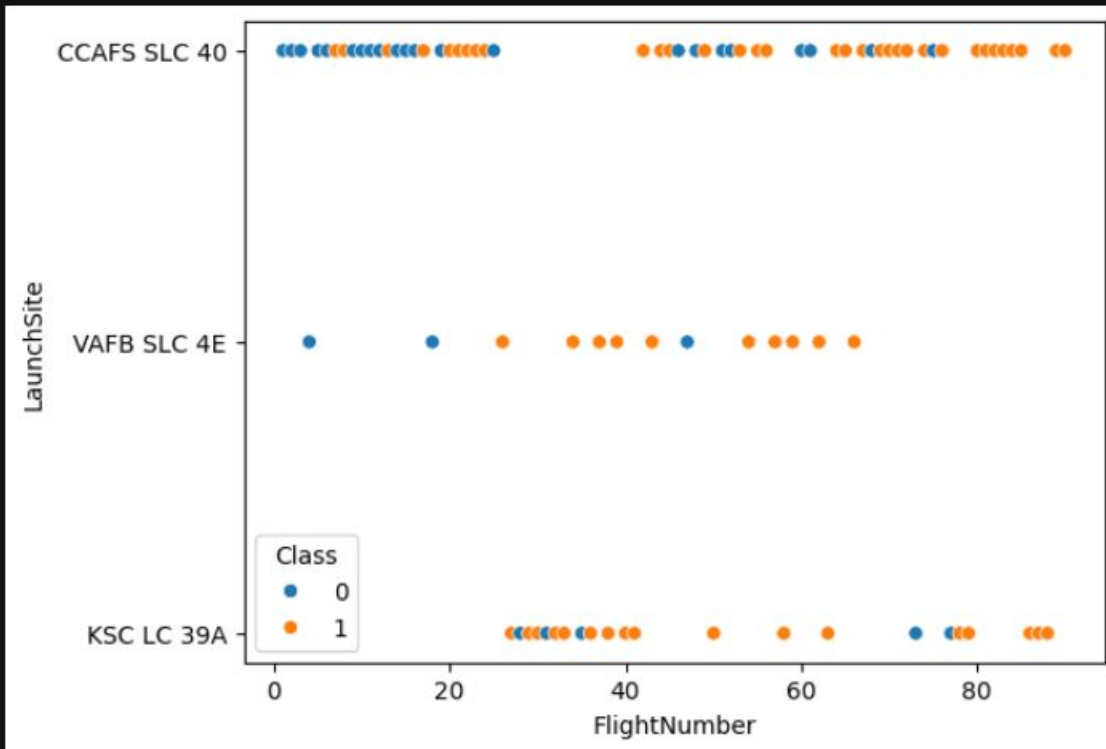


TASK2: EDA with Visualization Lab Example

Outputs

```
[6]: # Plot a scatter point chart with x axis to be Flight Number and y axis to be the Launch site, and hue to be the class  
sns.scatterplot(x='FlightNumber', y='LaunchSite', data=df, hue='Class')
```

```
[6]: <AxesSubplot:xlabel='FlightNumber', ylabel='LaunchSite'>
```



Link:

<https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/edadataviz.ipynb>

TASK2: EDA with Visualization Lab Example

Outputs

Create dummy variables to categorical columns

```
dh=pd.get_dummies(df['Orbit'],prefix='O_')df=pd.concat((dh,df)
```

.....

```
[27]:
```

	s_B0003	s_B0005	s_B0007	s_B1003	s_B1004	s_B1005	s_B1006	s_B1007	s_B1008	s_B1010	...	s_B1048	s_B1049	s_B1050	s_B1051	s_B1054	s_B1056	s_B1058	s_B10...
0	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	Fal
0	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	Fal
0	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	Fal
0	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	Fal
0	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	Fal
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
85	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	Fal
86	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	True	Fal
87	False	False	False	False	False	False	False	False	False	False	...	False	False	False	True	False	False	False	Fal
88	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	Fal
89	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	Fal

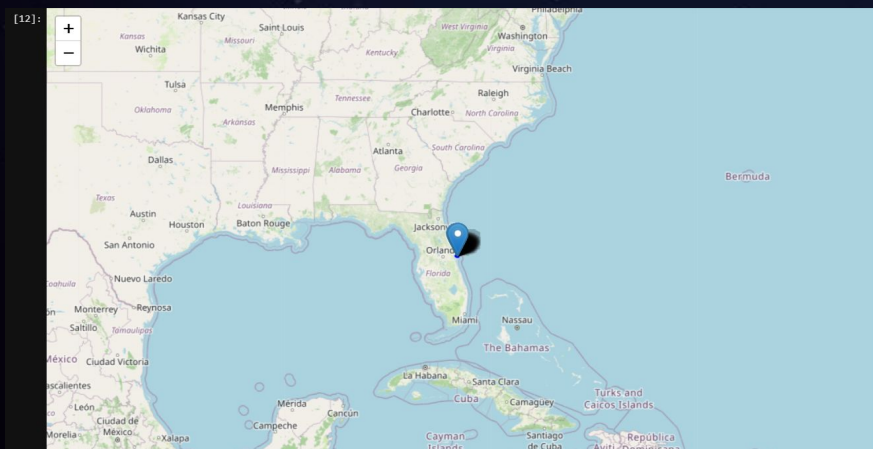
Link:

<https://github.com/Selmabanzem/a/Data-Science-Capston-Project/blob/main/edadataviz.ipynb>

TASK3: EDA with Visualization Lab Folium

The goal of this analysis was to investigate the geographical locations of SpaceX launch sites and identify possible patterns related to launch success. The analysis used **Folium** to create interactive maps showing the different launch sites and their locations. The launches were then visualized according to their **success or failure**, allowing the distribution of successful and unsuccessful launches at each site to be examined. The analysis also explored the **distances between launch sites and nearby geographical features**, such as coastlines, cities, roads, and other points of interest. Overall, the work provides a geographical perspective on SpaceX launch performance and helps identify factors and patterns that may be associated with the location of launch sites.

Here is example of one of the outputs



Link:

https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/lab_jupyter_launch_site_location.ipynb

TASK3: EDA with Visualization Lab Dash

The goal of this project was to create an interactive dashboard for analyzing SpaceX launch records and exploring the factors associated with launch success. The dashboard allows users to select a specific **launch site** or view data from all sites, with a pie chart showing the number of successful launches or the success and failure rate for a selected site. A **payload mass range slider** allows users to filter launches based on payload weight, while an interactive scatter plot displays the relationship between **payload mass and launch outcome**, with different booster categories shown for comparison. Overall, the dashboard provides an interactive way to explore SpaceX launch performance across different launch sites, payload ranges, and booster versions.

For the **pie chart**, KSC LC-39A

KSC LC-39A recorded 10 successful landings out of 13 launches, corresponding to a success rate of approximately 76.9%. This demonstrates comparatively strong landing performance for this launch site in the dashboard dataset.

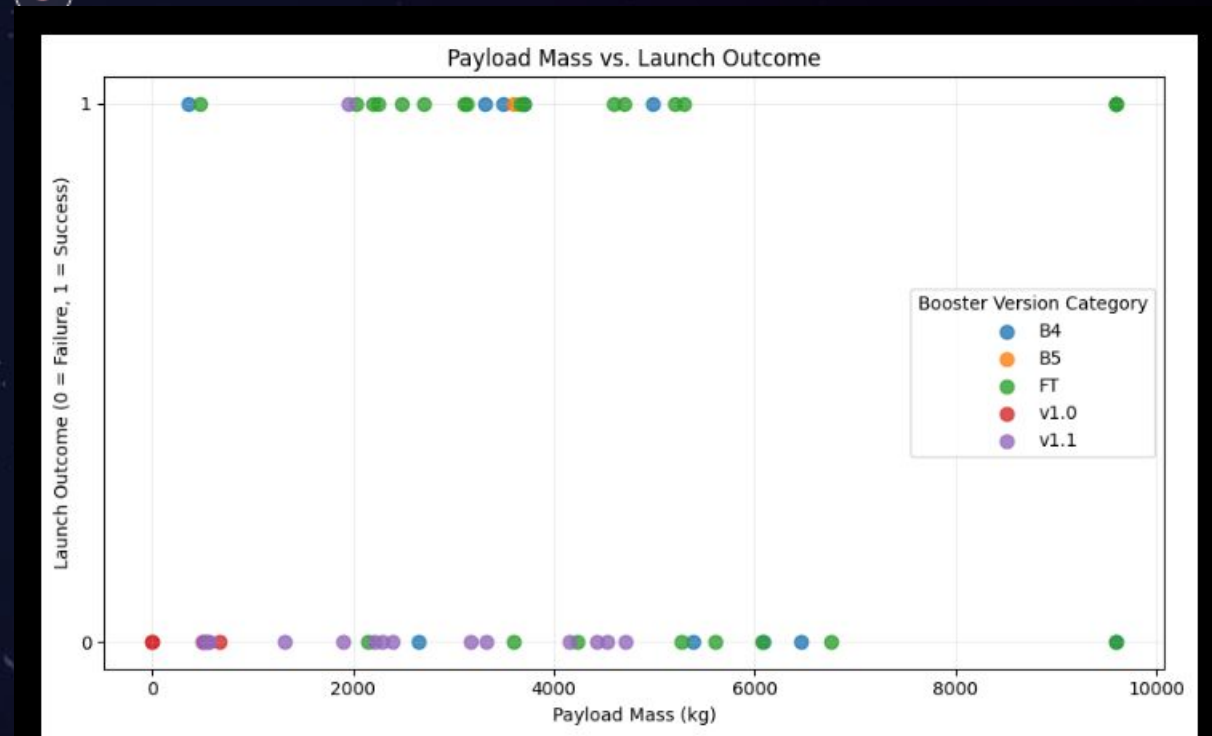
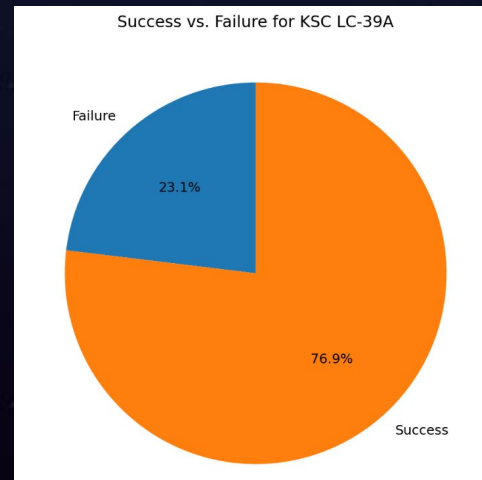
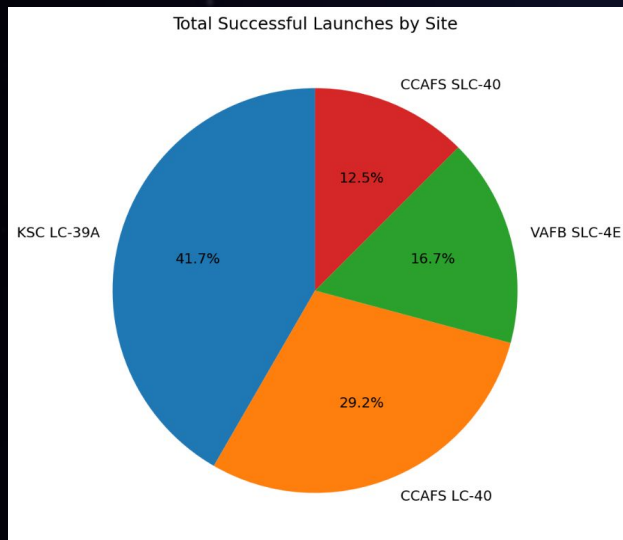
For the **payload scatter plot**,

The payload analysis shows that successful landings occur across different payload masses. The visualization also suggests that payload mass alone does not explain landing success, because outcomes vary across payload ranges and booster categories.

Link:

<https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/creating%20a%20dash.py>

TASK3: EDA with Visualization Lab Dash



TASK4: Predictive Analysis



Machine Learning Model and Results

The goal of the machine learning stage was to develop a classification model capable of predicting whether the **Falcon 9 first stage would successfully land**. After preparing the dataset, categorical variables were transformed into numerical features and the data was divided into training and testing sets. Several classification algorithms were then applied, including **Logistic Regression, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN)**. Hyperparameter tuning was performed using **GridSearchCV** to identify suitable parameter combinations and improve the predictive performance of each model.

Model Evaluation

The models were evaluated using several complementary metrics rather than relying only on accuracy. **Accuracy** measured the overall proportion of correct predictions, while the **F1 score** provided a balance between precision and recall. The **Jaccard score** measured the similarity between the predicted and actual successful-landing classes, and the **confusion matrix** provided a more detailed view of correct and incorrect predictions. Comparing these metrics made it possible to examine not only how often each model was correct, but also how effectively it identified successful and unsuccessful landings.

The evaluation shows that machine learning can capture meaningful relationships between launch characteristics and Falcon 9 landing outcomes. The differences between the models demonstrate why model comparison is important: a model with a strong overall accuracy may still produce different types of errors from another model. The tuned models therefore provide a more reliable basis for selecting the final prediction approach than using default model settings alone.

Link:

[https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

Link:

[https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

TASK4: Predective Anyalsis and Model Selection Outputs

Fit and tranform the data :

Split into train and test and then train
it on logistic regression first

TASK 1

Create a NumPy array from the column `Class` in `data`, by applying the method `to_numpy()` then assign it to the variable `Y`, make sure the output is a Pandas of column`Y`).

```
[12]: y=data['Class'].to_numpy()
```

TASK 2

Standardize the data in `X` then reassign it to the variable `X` using the transform provided below.

```
[13]: # students get this
transform = preprocessing.StandardScaler()
x_fitted=transform.fit_transform(X)
```

TASK 3

Use the function `train_test_split` to split the data `X` and `Y` into training and test data. Set the parameter `test_size` to 0.2 and `random_state` to 2. The training data and test data should the following labels.

```
X_train, X_test, Y_train, Y_test
```

```
[14]: X_train, X_test, Y_train, Y_test=train_test_split(x_fitted,y,random_state=2,test_size=0.2)
```

we can see we only have 18 test samples.

```
[15]: Y_test.shape
```

```
[15]: (18,)
```

TASK 4

Create a logistic regression object then create a GridSearchCV object `logreg_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
[16]: parameters ={'C':[0.01,0.1,1],
                  'penalty':['l2'],
                  'solver':['lbfgs']}
```

```
[17]: parameters =({'C':[0.01,0.1,1], 'penalty':['l2'], 'solver':['lbfgs']})# L1 Lasso L2 ridge
lr=LogisticRegression()
lr.fit(X_train,Y_train)
y=lr.predict(X_test)
```

Link:

[https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

TASK4: Predective Anyalsis and Model Selection Outputs

Fit and tranform the data :

Split into train and test and then train
it on logistic regression first

TASK 1

Create a NumPy array from the column `Class` in `data` , by applying the method `to_numpy()` then assign it to the variable `Y` ,make sure the output is a Pandas of column'].
[12]: `y=data['Class'].to_numpy()`

TASK 2

Standardize the data in `X` then reassign it to the variable `X` using the transform provided below.

[13]: `# students get this
transform = preprocessing.StandardScaler()
x_fitted=transform.fit_transform(X)`

TASK 3

Use the function `train_test_split` to split the data `X` and `Y` into training and test data. Set the parameter `test_size` to 0.2 and `random_state` to 2. The training data and test data should the following labels.

`X_train, X_test, Y_train, Y_test`

[14]: `X_train, X_test, Y_train, Y_test=train_test_split(x_fitted,y,random_state=2,test_size=0.2)`

we can see we only have 18 test samples.

[15]: `Y_test.shape`

[15]: `(18,)`

TASK 4

Create a logistic regression object then create a GridSearchCV object `logreg_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters` .

[16]: `parameters ={'C':[0.01,0.1,1],
 'penalty':['l2'],
 'solver':['lbfgs']}`

[17]: `parameters ={'C':[0.01,0.1,1], 'penalty':['l2'], 'solver':['lbfgs']}]# L1 Lasso L2 ridge
lr=LogisticRegression()
lr.fit(X_train,Y_train)
y=lr.predict(X_test)`

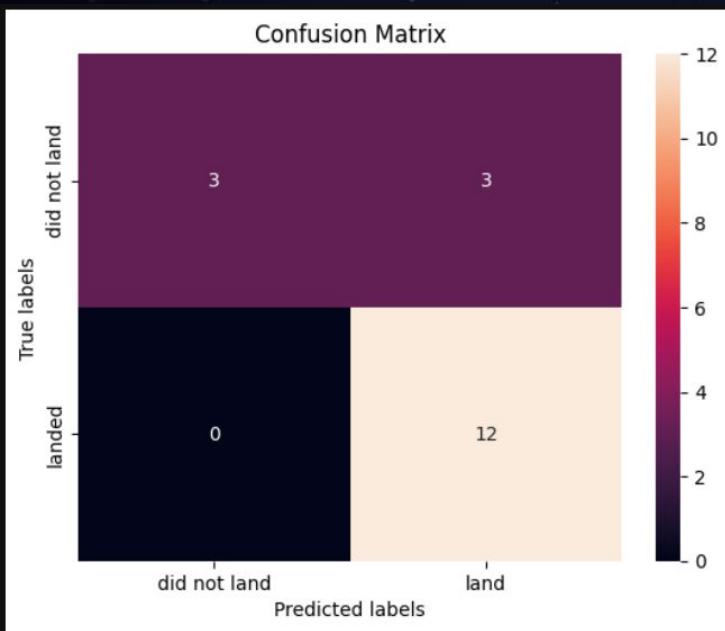
TASK4: Predictive Analysis Outputs

1- Logistic regression:

1- we have worked on it and the score is : 1 ,

accuracy : 0.8342857142857143

The confisoun matrix :



Link:

[https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

Link:

[https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

TASK4: Predictive Analysis Outputs

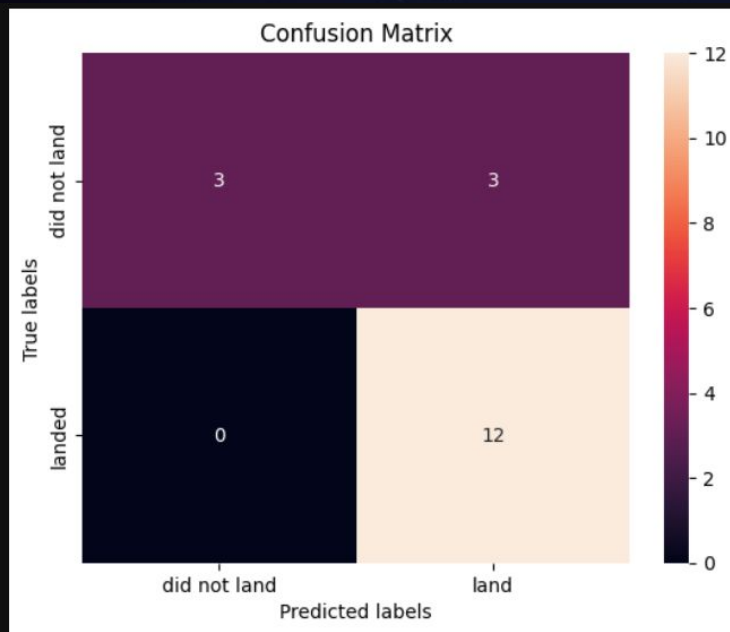
1- SVM:

tuned hpyerparameters

:(best parameters) {'C': 0.03162277660168379, 'gamma': 0.001, 'kernel': 'linear'}

accuracy : 0.8342857142857142

the photo for the confusion matrix:



TASK 6

Create a support vector machine object then create a `GridSearchCV` object `svm_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
[23]: parameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
                  'C': np.logspace(-3, 3, 5),
                  'gamma':np.logspace(-3, 3, 5)}
      svm = SVC()

[24]: svm_cv=GridSearchCV(svm,param_grid=parameters)
      svm_cv.fit(X_train, Y_train)
      print("tuned hpyerparameters :(best parameters) ",svm_cv.best_params_)
      print("accuracy :",svm_cv.best_score_)

tuned hpyerparameters :(best parameters) {'C': 0.03162277660168379, 'gamma': 0.001, 'kernel': 'linear'}
accuracy : 0.8342857142857142

[25]: print("tuned hpyerparameters :(best parameters) ",svm_cv.best_params_)
      print("accuracy :",svm_cv.best_score_)

tuned hpyerparameters :(best parameters) {'C': 0.03162277660168379, 'gamma': 0.001, 'kernel': 'linear'}
accuracy : 0.8342857142857142
```

TASK4: Predictive Analysis Outputs

Link:

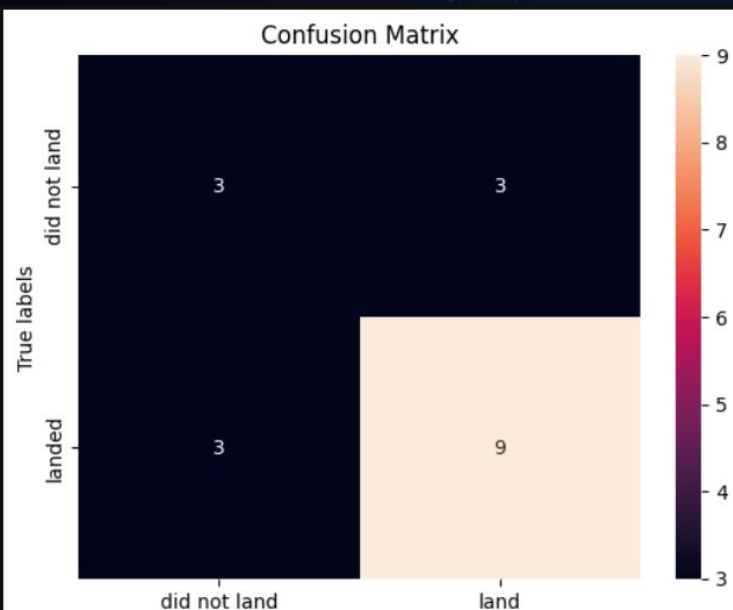
[https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

1- TREE:

tuned hpyrparameters :(best parameters) {'criterion': 'entropy', 'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 5, 'splitter': 'best'}

accuracy : 0.8607142857142858

, the photo for the confusion matrix:



TASK 9

Calculate the accuracy of tree_cv on the test data using the method `score` :

```
[47]: tree.fit(X_train,Y_train)
      tree.score(X_train,Y_train)
```

```
[47]: 1.0
```

```
[48]: test_accuracy = tree.score(X_test, Y_test)
      print("Test Accuracy:", test_accuracy)
```

```
Test Accuracy: 0.8333333333333334
```

We can plot the confusion matrix

```
[40]: yhat = tree_cv.predict(X_test)
      plot_confusion_matrix(Y_test,yhat)
```

Link:

[https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

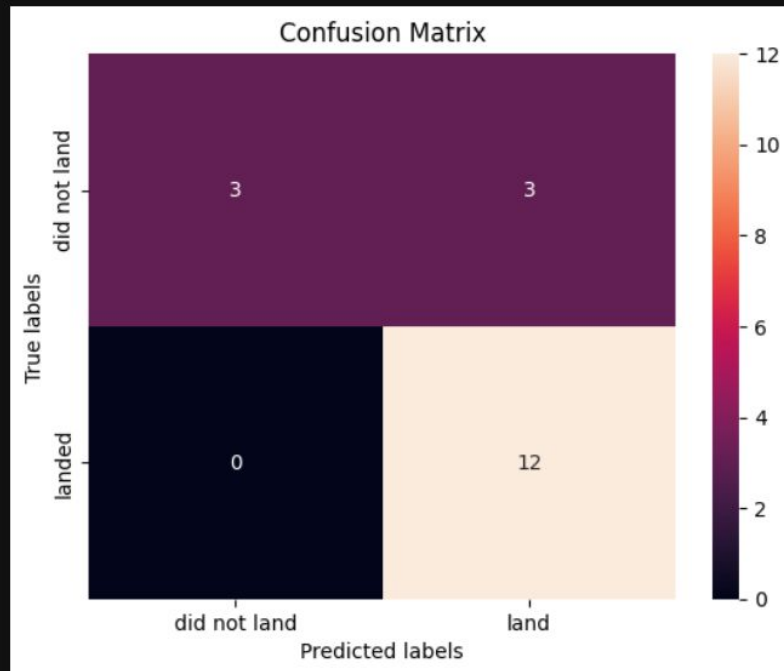
TASK4: Predective Anaylsis Outpusts

1- knn:

tuned hpyerparameters :(best parameters) {'algorithm': 'auto', 'n_neighbors': 8, 'p': 1}

accuracy : 0.8609523809523811

, the photo for the confusion matrix:



TASK4: Predictive Analysis Output

Link:

[https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

Model Selection

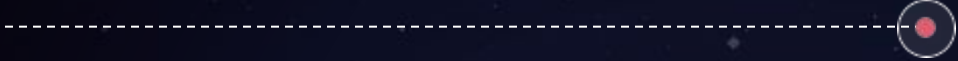
After training and tuning four different classification algorithms—K-Nearest Neighbors (KNN), Decision Tree, Support Vector Machine (SVM), and Logistic Regression—their performances were compared using classification accuracy. The **KNN model achieved the highest accuracy of approximately 86.10%**, using the optimized hyperparameters `n_neighbors = 8`, `p = 1`, and `algorithm = 'auto'`. The **Decision Tree produced a very similar accuracy of approximately 86.07%**, with its best configuration including `criterion = 'entropy'`, `max_depth = 10`, `max_features = 'sqrt'`, `min_samples_split = 5`, and `splitter = 'best'`. In comparison, both **SVM and Logistic Regression achieved lower accuracies of approximately 83.43%**.

Based on accuracy alone, **KNN would be selected as the final model because it achieved the highest predictive performance among the evaluated algorithms**. However, the difference between KNN and the Decision Tree is extremely small—approximately **0.02 percentage points**—so the two models should be considered practically very close in performance. Therefore, accuracy alone does not provide strong evidence that KNN is substantially better than the Decision Tree. Additional evaluation metrics such as **precision, recall, F1-score, confusion matrix, and cross-validation performance** should also be considered before making the final deployment decision, particularly if the classes are imbalanced.

The selected KNN configuration uses **8 nearest neighbors**, meaning that predictions are determined from the eight most similar observations in the training data. The parameter `p = 1` corresponds to the **Manhattan distance**, which calculates similarity based on the sum of absolute differences between feature values. Since KNN is a distance-based algorithm, **feature scaling is particularly important**; therefore, numerical features should be standardized or normalized before training and prediction.

Overall, **KNN is the preferred model based on the current evaluation results**, since it provides the highest observed accuracy of **86.10%**. Nevertheless, because the Decision Tree achieves almost identical performance, the final choice could also depend on practical requirements. If maximizing the measured predictive accuracy is the primary objective, KNN is the appropriate choice. If model interpretability and the ability to explain individual decision rules are important, the Decision Tree may be preferable despite its marginally lower accuracy. Thus, the current results support **KNN as the final predictive model while identifying the Decision Tree as a very competitive alternative**.

TASK4: Predictive Analysis



Best Model Application

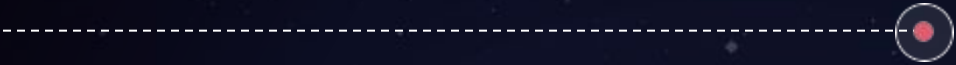
After comparing the trained models, the model selected from the notebook can be applied to **new, previously unseen launch conditions**. Given information such as launch site, payload characteristics, orbit, booster configuration, and other engineered features, the model produces a predicted landing outcome. This transforms the analysis from simply describing historical SpaceX launches into a practical predictive tool.

The most valuable aspect of this application is its potential decision-support role. Before a future launch, the model could provide an additional data-driven estimate of first-stage landing success based on characteristics similar to previous missions. This could help analysts investigate which combinations of launch conditions are associated with higher or lower predicted success.

Link:

[https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/Selmabanzema/Data-Science-Capston-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

CONCLUSION AND INSIGHT



Conclusion and Innovative Insight

Overall, the machine learning analysis demonstrates how historical SpaceX launch data can be transformed into a predictive framework for Falcon 9 landing success. The project combines **data preparation, feature engineering, multiple classification algorithms, hyperparameter optimization, and model evaluation** to move from raw launch records toward predictive insight.

A particularly interesting insight is that the prediction should not be interpreted as a simple question of whether a rocket is “good” or “bad.” Landing success is influenced by a **combination of operational factors**, meaning that the same payload mass or booster type can have different implications under different launch conditions. This suggests a more innovative direction for future work: instead of using the model only as a yes/no predictor, it could be developed into a **mission-risk analysis tool** that estimates the probability of landing success under different combinations of launch site, payload, orbit, and booster characteristics. Such a system could allow analysts to compare hypothetical mission configurations before launch and identify conditions associated with different predicted outcomes.

The final value of the project therefore lies not only in predicting a landing outcome, but in demonstrating how machine learning can convert historical launch data into **actionable, scenario-based insights for future missions**.

The Hole Project Link

<https://github.com/Selmabanzema/Data-Science-Capston-Project->